SDE Intern – AI Agents & Automation · Hidani Tech

Job Chat Assistant

This is a chat that reads a resume, recommends roles from a file of real job postings, and saves an application when you ask. There is no login. The listings are not typed in by hand. They come from a JSON file a scraper built by calling public job-board APIs.

Live chat. https://job-assitant.vercel.app/

Source code. https://github.com/kshitiz-12/job_assitant

The dashboard is on Vercel. The API behind it is https://job-assitant.onrender.com. The names are spelled job-assitant and job_assitant, without the second s. Adding that s will 404. I checked /api/health on 21 October 2026. It came back ok: 1,970 jobs, 1,600 from Greenhouse, 250 from Ashby, and 120 from Lever. The file was generated on 19 October 2026 at 18:58 UTC. The free Render instance sleeps, so the first message after a quiet stretch can take half a minute while the host wakes up.

What the assignment asked for

Four things, and where each one is.

- A working chat: https://job-assitant.vercel.app/. Upload a PDF or DOCX and talk to it.
- The public repo, with the full source: https://github.com/kshitiz-12/job_assitant. Default branch is main.
- The job database is data/jobs.json. The script that built it is scripts/scrape_jobs.py. There is no second, hand-written list of jobs.
- README.md at the repo root covers setup, the agent split, the data flow, and the APIs. This note says the same things in plain sentences. If you are running the project, follow the README.

What a conversation does

The page opens on an empty thread. Across the top it shows how many roles are loaded, how many boards they came from, and how many applications this browser has saved. Those numbers come from the health check and the session.

You drop in a PDF or a DOCX. The assistant reads it back: name, contact, title, skills, experience, education, and projects. Then it shows roles, five at a time. Each card has a reason, a score, and a link to the original posting. You stay in the same thread after that.

- “show only remote roles”, “jobs needing React”, “in Berlin”, “intern roles”, “show more”.
- “apply to job 3” records that card. Asking again does not write a second row.
- “show my applied jobs” lists the job id, the date, and the status.
- A bad parse can be corrected in the same chat: “my title is Data Analyst”, “I have 4 years”, “I know Figma”.

The chips under a reply only send a sentence you could have typed. Matching and applying still go through the same chat turn.

Apply does not submit the company's own form. Those forms live on the employer's site, and a lot of them ask for a captcha. The chat writes a row and keeps the link. Status stays not_submitted until someone finishes the form on the company site. I would rather say that plainly than pretend a button click was an application.

The repository

The repo is https://github.com/kshitiz-12/job_assitant. Python 3.12 runs the API. The dashboard is React 19 and Vite. They deploy as two services and only meet over HTTP.

- app/main.py is the HTTP layer: health, session, one job, and the chat turn.
- app/agents/orchestrator.py reads the turn and calls the specialist the plan names.
- app/agents/intents.py decides which tool to run, from phrases a person actually types.
- app/agents/resume.py turns a PDF or DOCX into the profile the chat shows back.
- app/agents/matcher.py filters, scores, and writes a reason that only cites what was compared.
- app/agents/applications.py resolves "job 3" and writes one row per person per job.
- app/agents/advice.py suggests resume edits from the roles just shown.
- app/ingest.py turns three different job-board payloads into one shape. It does not touch the network.
- app/store.py loads the JSON files and keeps sessions.
- scripts/scrape_jobs.py builds data/jobs.json.
- frontend/ is the dashboard. On Vercel the project root is this folder.
- tests/ runs with pytest. scripts/make_sample_resume.py builds the resume the tests use.

Chat state sits in data/sessions.json, which is gitignored. Saved applications sit in data/applications.json. On the free Render plan the disk does not outlive the process, so a saved application lasts until that instance sleeps or restarts. jobs.json is in the repo and is read when the server starts, so the listings come back.

The job file

data/jobs.json is about 3.6 MB. The wrapper is short: generated_at, count, sources, a one-line note, and the jobs array. Count is 1,970. They come from 135 companies. 518 are marked remote. 1,409 rows had at least one skill pulled out of the posting. 48 titles contain "intern".

Every row has the same fields: id, title, company, location, experience_required, experience_years_min, skills, description, source_link, source, and remote. The description is a plain-text excerpt, clipped around 1,400 characters, not the full HTML page. source_link is the original posting.

The first row in the current file is gh_6sense_8164298, Business Development Representative at 6sense, with a link back to boards.greenhouse.io. Ids are prefixed so the three feeds cannot collide: gh_ for Greenhouse, lv_ for Lever, ab_ for Ashby, then the board token and the employer's own id.

A lot of companies show up with exactly 26 roles. That is the scraper's per-board cap, not a claim that the company only has 26 openings. The newest postings on that board are the ones kept, so one employer cannot fill the file.

How the file was scraped

scripts/scrape_jobs.py does not crawl HTML and it does not log into anything. It downloads three public CSVs of board slugs, keeps boards whose open-role count sits in a sane range, then calls each employer's own job-board API. Those are the feeds behind the public careers pages.

The directory is kalebconfer-sys.github.io/job-board-directory: greenhouse.csv, ashby.csv, and lever.csv. The script does not guess tokens. A short fallback list of known boards is only used if that CSV cannot be downloaded, and it is not big enough to reach 1,500 jobs. If that happens the script says so and exits, unless you pass --allow-short while debugging.

- Greenhouse: GET https://boards-api.greenhouse.io/v1/boards/{token}/jobs?content=true
- Lever: GET https://api.lever.co/v0/postings/{slug}?mode=json
- If the US Lever host returns 404, the same path is tried on api.eu.lever.co.
- Ashby: GET https://api.ashbyhq.com/posting-api/job-board/{name}

None of those GETs need a key. The script retries a few times on a timeout or a 429, skips a board that errors, and only runs a handful of boards at once. Greenhouse carries most of the volume, up to 1,600. Ashby is capped at 250 and Lever at 120, so the file is not one vendor's idea of a job. The whole file stops at 2,000. The default target is 1,850. This run landed at 1,970.

Duplicates drop out twice: once on the posting URL, and again on company plus title plus location. app/ingest.py is where a Greenhouse payload, a Lever payload, and an Ashby payload become the same record. The scraper does the HTTP. Ingest stays pure, so a bad posting can be tested without the network. Ashby listings marked isListed false are skipped.

From the repo root:

```
python scripts/scrape_jobs.py
python scripts/scrape_jobs.py --target 1800 --out data/jobs.json
```

Re-run it when you want a fresher snapshot. Live postings move. The checked-in file is the snapshot from 19 October 2026, not a live query on every chat turn. The chat is fast because it reads the file.

Setup

Python 3.12. Two processes: the API on port 8000, and the Vite dev server, which forwards /api to that port. The README shows the Windows activate path. On macOS or Linux, source the bin script instead.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python scripts/scrape_jobs.py
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

On Windows, activate with .venv\Scripts\activate. In a second terminal:

```
cd frontend
npm install
npm run dev
```

Open http://127.0.0.1:5173 and upload a resume. Tests:

```
pip install -r requirements-dev.txt
pytest
```

Looser sentences, such as “only frontend roles” or “remote React jobs in Berlin”, go through Gemini if you copy .env.example to .env and set GEMINI_API_KEY. The default model is gemini-2.5-flash. It classifies the sentence and can rephrase the reply. The roles still come from jobs.json. Without a key, the built-in phrases still work. Apply and “show my applied jobs” stay on those exact phrases even when a key is set, so a loose sentence cannot invent an application. Do not commit the key. On Render it belongs in the service environment.

Deploy is split the same way as local. The API is a Render web service from render.yaml: install the requirements, start uvicorn, health check at /api/health. The frontend is a Vite project on Vercel with its root set to frontend. VITE_API_URL is the Render origin with no trailing slash, https://job-assitant.onrender.com. Vite bakes that value in at build time, so the frontend has to be redeployed if the API origin changes. There is also a Dockerfile if you want the API in a container.

How the agent is wired

One orchestrator. It does not score jobs and it does not write the application file. It asks interpret() for a small plan, calls the specialist that plan names, and then builds a reply from what came back. The activity line in the chat, “Read the resume · Compared the profile with N listings”, is that plan. It is not a hidden chain of thought.

A message or a resume file lands on the orchestrator. The plan then goes to one of three specialists. The resume agent writes the session profile. The match agent reads data/jobs.json. The application agent writes data/applications.json. A smaller advice path only comments on the profile and the roles just shown. It does not invent employers or metrics.

Resume agent. pypdf for PDF, python-docx for DOCX, tables included. A scan with no text layer, a password-protected PDF, or a file that is not a resume comes back as a normal chat message. It would rather say it could not read the file than invent a name. If you already had a resume and the new upload looks wrong, the old profile stays. You can confirm a doubtful file with “yes, use this file”.

Match agent. Filters run first: remote, a required skill, a city, a company, intern, junior, senior, or a track such as frontend. Then each remaining posting is scored as 52% skill overlap, 30% title similarity, and 18% years. Skills that show up on almost every posting, such as git, agile, and HTML, barely move the score. A resume with a few years is nudged down on staff and principal roles so they do not win just for sharing a language. If you did not ask for something specific and the best score is weak, the page shrinks to three cards instead of five. One employer is capped at two cards on that page, on top of the scraper’s 26-per-board cap. A reason only

mentions skills, a title, and years that were actually compared, so the sentence cannot invent a technology the resume does not have.

Application agent. “job 3”, “the second one”, or a pasted job id is resolved against the list just shown. One row per person per job. Until the resume has an email, applications are keyed by the browser session. After that they follow the email, so uploading the same resume again still finds them. That is a stable key, not an account.

Advice. “What should I change on my resume?” looks at the roles just shown and names skills those postings ask for that the profile does not have. It tells you to add a bullet only where you have actually used the tool. If the skills already overlap, it says to rewrite the result, not to pad the tool list.

If GEMINI_API_KEY is set, the model may choose the tool for an ordinary sentence and rewrite the answer in the tone of the thread. A missing key, a timeout, or a reply that claims the application was submitted falls back to the local sentence. The dataset still picks the jobs. The demo works with no API key.

Data flow

A posting becomes a card in five steps. Nothing in the middle goes out to an employer.

- The scraper downloads the board directory, then each employer’s job-board API, and writes data/jobs.json.
- app/ingest.py normalizes those payloads into one record: id, title, company, location, experience, skills, a plain-text excerpt, the source link, plus source, remote, and experience_years_min.
- On startup the server loads the file into memory and keeps a skill set and a search blob on each row, so a chat turn does not re-parse HTML.
- POST /api/chat updates the session, and maybe applications.json, and returns the reply plus the cards.
- The browser renders the cards. A refresh reloads the session from GET /api/session/{id}, which is why the thread survives a reload in the same browser.

The frontend stores the session id in localStorage under job-chat-session. “New chat” drops it. Uploads are capped at 4 MB. An empty file, a message over 20,000 characters, or a turn that throws is answered in the thread, not with a stack trace. Mistakes that belong in the conversation, such as an unreadable resume, no matches, “job 9” when the list has five, or a duplicate application, come back as the assistant’s reply with HTTP 200.

The HTTP API

The dashboard talks to four routes. Chat is multipart form data, because a turn might include a file.

- POST /api/chat. Fields are message, session_id, and an optional file. The body comes back with reply, jobs, profile, filters, saved applications, activity, and suggestions.
- GET /api/health. Returns ok, the job count, the source breakdown, and generated_at.

- GET /api/session/{id}. Returns the profile, the transcript, the filters, and that person's applications. Send back the id from the previous response. The first turn can leave session_id empty.
- GET /api/jobs/{job_id}. Returns one stored posting, or 404 if it is not in the file.

message can be empty when a file is attached. A saved application in the file looks like this, with the applicant key, name, title, company, and source link stored alongside so "show my applied jobs" can name the role:

```
{
  "job_id": "gh_gitlab_123",
  "applied_date": "2026-10-19",
  "status": "not_submitted"
}
```

APIs the project calls

Job data needs no keys. The optional language call is the only one that does, and the product runs without it.

- Job board directory. Public CSV of Greenhouse, Ashby, and Lever board slugs. No auth.
- Greenhouse Job Board API. Public careers feed for one board token. No auth on GET.
- Lever Postings API. Public postings for one slug. The EU host is used if the US one returns 404. No auth.
- Ashby posting API. Public job board for one Ashby name. No auth.
- Gemini generateContent, optional. Classifies a loose sentence and phrases the reply. Default model is gemini-2.5-flash. Needs GEMINI_API_KEY.

The scraper identifies itself as JobChat/1.0 and backs off when a host is unhappy. It does not walk a page that asks not to be walked, and it does not touch an employer's authenticated apply endpoint. The product stops at the saved row plus the link.

What to try first

Open https://job-assitant.vercel.app/. If the numbers along the top stay on a dash for a bit, wait for Render to wake, then refresh. You should see 1,970 roles. Upload a resume of your own, or generate the one the tests use with scripts/make_sample_resume.py. Ask for remote roles, apply to job 2, ask again, and then ask to see applied jobs. The second apply should not add a row.

For the code, clone https://github.com/kshitiz-12/job_assitant and start with README.md, then scripts/scrape_jobs.py, then app/agents/orchestrator.py. Those three files are the setup, the job file, and the way a sentence becomes a card.

Job file snapshot: 19 October 2026, 18:58 UTC. Live health check: 21 October 2026. Chat https://job-assitant.vercel.app/ · API https://job-assitant.onrender.com/api/health · source https://github.com/kshitiz-12/job_assitant.